GUEST SESSION

Modern Python

uv, Readable Code, and AI Agents

A hands-on refresher

Somto Muotoe

October 2026

`github.com/smuotoe/mcda-python-refresher`

Install uv now

Do this while everyone settles in. It takes about a minute.

Windows (recommended)

```
winget install --id=astral-sh.uv -e
```

Asked to accept the source agreements? Type `Y`. "winget" not recognised? Install **App Installer** from the Microsoft Store.

macOS / Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Or, with Homebrew: `brew install uv`

Then open a NEW terminal

```
uv --version
git --version
```

No git? You can still get the exercises later: on GitHub, **Code > Download ZIP**.

Windows fallback (PowerShell installer):
`powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`

Last resort (Python already installed): `python -m pip install uv`, then type `python -m uv` wherever the slides say `uv`.

Sources: docs.astral.sh/uv/getting-started/installation

ABOUT ME

Hi, I'm Somto

Senior ML Engineer

Agentic AI risk and compliance

In my spare time: cycling and chess

Agenda

0:05 – 0:30 · Part 1

uv

Why Python packaging was a mess, and the one tool that fixed it

0:30 – 1:00 · Part 2

Code you can read

Functions, type hints, docstrings and classes. Fill in code, then review an AI's

1:00 – 1:25 · Part 3

How code gets written today

Ruff, type checking, and AGENTS.md: rules your AI agent follows

1:25 – 1:30

Wrap-up

Checklist, resources, questions

Laptops open the whole time. Setup steps, exercises and a uv cheat sheet are in the repo README.

Quick poll: show of hands

How do you manage Python environments today?

- pip + venv
- conda
- poetry
- uv
- whatever works

Do you use AI to write code?

- chat (ChatGPT, Claude, ...)
- autocomplete (Copilot, ...)
- agents (Claude Code, Codex, ...)
- not yet

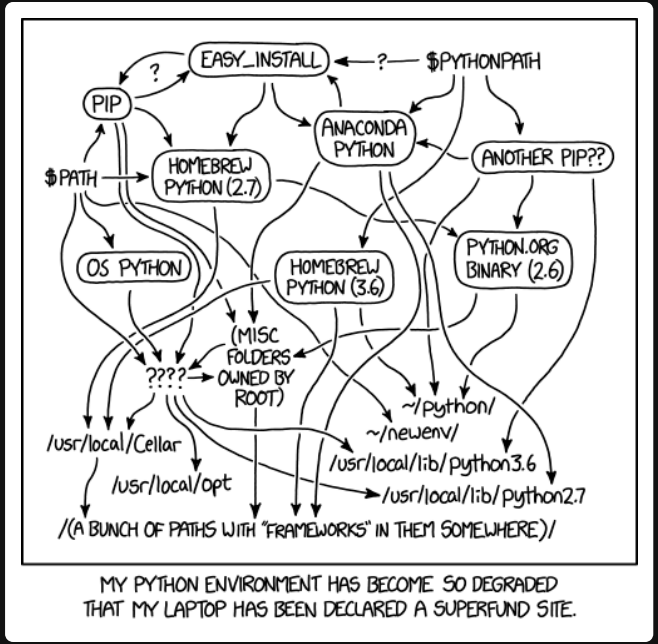
No wrong answers. "Whatever works" is the most honest one.

PART 1 · 0:05 - 0:30

uv: one tool for Python

Python packaging used to be a mess. Here's what fixed it.

Python packaging used to be a mess



xkcd #1987, "Python Environment" (2018)

"Which pip is this?" `pip install` went into a different Python than the one running your code.

Forgot to activate the venv? You just installed into your system Python.

"Works on my machine." `requirements.txt` without exact versions, so everyone got something different.

A tool for every job: pip, venv, pip-tools, pyenv, pipx, Poetry, conda. Each fixed one problem.

Slow. Installing a data science stack meant waiting minutes.

Sources: xkcd.com/1987 (CC BY-NC 2.5)

What uv replaces

You used to need	With uv
pip	`uv add`, `uv pip install`
pip-tools	`uv lock`, `uv sync`
venv / virtualenv	automatic (or `uv venv`)
pipx	`uvx`, `uv tool install`
pyenv	`uv python install`
poetry	`uv init`, `uv add`, `uv lock`

One fast binary

Written in Rust by Astral, the makers of the Ruff linter. It doesn't depend on which Python you have.

10-100x

faster than pip, according to Astral's docs. Let's test that.

Sources: docs.astral.sh/uv · astral.sh

The race: pip vs uv

```
uv run demos/race.py
```

Two fresh environments. pip and uv install the same packages **at the same moment**:

```
jupyterlab pandas scikit-learn matplotlib
```

106 packages once dependencies are counted.

`uv run demos/race.py --cold` makes both ignore their cache: your first-ever install.

`uv run demos/race.py requests flask` races your own packages.

Both tools keep a cache. So why is uv faster?

Step	pip	uv
download	skipped (cached)	skipped (cached)
pick versions	in Python	in Rust, in parallel
install files	unzip and copy every file, every time	link files it already unpacked

With both caches warm, the network is out of the picture. What's left is the tool.

Sources: docs.astral.sh/uv/concepts/cache · pip: caching

A uv project in four commands

```
uv init --no-package my-app
cd my-app
uv add pandas
uv run main.py
```

```
my-app/
├── .python-version   which Python this project uses
├── main.py
├── pyproject.toml    what you asked for
├── uv.lock           exactly what you got
└── .venv/            created for you; never edit it
```

Also a `README.md`, `.gitignore` and a git repo.

```toml
# pyproject.toml, after uv add pandas
[project]
name = "my-app"
version = "0.1.0"
requires-python = ">=3.13"
dependencies = [
    "pandas>=3.0.6",
]
```

No activation, ever. `uv run` creates `.venv`, syncs it with `uv.lock`, then runs your code. No more "which pip?"

`uv remove pandas`: take it out again

`uv sync`: make `.venv` match `uv.lock` exactly

Sources: docs.astral.sh/uv/concepts/projects/init · docs.astral.sh/uv/concepts/projects/dependencies

Scripts that bring their own dependencies

```python
# /// script
# requires-python = ">=3.12"
# dependencies = [
#     "pyfiglet>=1.0.4",
#     "rich>=15.0.0",
# ]
# ///
import pyfiglet
from rich.console import Console
from rich.panel import Panel

art = pyfiglet.figlet_format("uv", font="slant")
Console().print(Panel(art, border_style="green"))
```

```
uv init --script banner.py
uv add --script banner.py rich pyfiglet
uv run banner.py
```

The `# /// script` block lists what the script needs. `uv run` reads it, installs those packages into a cached environment of their own, and runs the script.

No project, no venv, no `pip install`. Send the one file to anyone with uv and it just works.

> **You've already seen one:** `demos/race.py` starts with this block. That's why it ran without installing `rich` first.

Sources: PEP 723 · docs.astral.sh/uv/guides/scripts

Your turn: hello, uv (5 min)

```
uv init --no-package hello-uv
cd hello-uv
uv add rich
# replace main.py with the code on the right
uv run main.py
uvx cowsay -t "uv rocks"
```

`uvx` runs a tool once, in a throwaway environment, without adding it to your project.

> **Bonus:** run `uv tree` to see what `rich` pulled in, then open `pyproject.toml` and `uv.lock`.

```python
from rich.console import Console
from rich.table import Table


def main() -> None:
    table = Table(title="My Python toolbox")
    table.add_column("Tool", style="cyan")
    table.add_column("What it does")
    table.add_row("uv", "projects and dependencies")
    table.add_row("ruff", "linting and formatting")
    table.add_row("ty", "type checking")
    Console().print(table)


if __name__ == "__main__":
    main()
```

uv cheat sheet

I want to...	Command
start a project	`uv init --no-package my-app`
run code	`uv run main.py`
add a dependency	`uv add httpx`
add a dev-only tool	`uv add --dev ruff`
remove a dependency	`uv remove httpx`
match the lock file	`uv sync`
upgrade one package	`uv lock --upgrade-package httpx`
see the dependency tree	`uv tree`
update uv	`uv self update` / `winget upgrade`

I want to...	Command
install a Python	`uv python install 3.13`
pin a project's Python	`uv python pin 3.13`
try another Python once	`uv run --python 3.12 ...`
write a one-file script	`uv init --script check.py`
give a script dependencies	`uv add --script check.py rich`
use a package once	`uv run --with rich ...`
run a tool once	`uvx ruff check .`
install a tool	`uv tool install ruff`
use a requirements.txt	`uv pip install -r requirements.txt`

Photograph this one. The repo README has the same cheat sheet.

Get the exercises

```
git clone https://github.com/smuotoe/mcda-python-refresher
cd mcda-python-refresher
uv sync
```

> **One command, my exact environment.** `uv sync` reads `uv.lock` and `.python-version`, and downloads Python 3.13 if you don't have it.

No git? On the GitHub page: **Code > Download ZIP**, unzip, and `cd` into the folder.

Open the folder in VS Code: `code .`

```
exercises/
  gradebook.py     Exercise 1
  review.py        Exercise 2
starter/
  AGENTS.md        Exercise 3
  scores.csv
demos/
  race.py
  banner.py
  type_hints.py
pyproject.toml
uv.lock
```

PART 2 · 0:30 - 1:00

Code you can read

Functions, type hints, docstrings and classes

AI writes the code. Why learn to read it?

You review it

AI output is a first draft. Someone has to decide whether it's right before it ships. That's you.

You debug it

When it breaks, the traceback points at a line of code. You need to know what that line does.

You own it

"The AI wrote it" is not an answer in a code review, an exam or an incident report.

If you can't explain it, you don't own it.

So today is about reading code: fill in a few gaps, then review code an AI wrote.

Functions: anatomy of a good one

```python
def average(scores: list[float], drop_lowest: bool = False) -> float:
    """Return the mean score, optionally ignoring the lowest one."""
    if not scores:
        raise ValueError("scores must not be empty")
    if drop_lowest and len(scores) > 1:
        scores = sorted(scores)[1:]
    return sum(scores) / len(scores)
```

Name says what it returns; **type hints** say what goes in and out.

Docstring: what it does, in one line or more.

One function, one job. Raise early on bad input.

Return values, don't print them, so other code can use the result.

Classic bug: a mutable default

```python
def add_tag(tag: str, tags: list[str] = []):
    tags.append(tag)
    return tags

add_tag("a")  # ['a']
add_tag("b")  # ['a', 'b']  shared list!
```

The `[]` is created once, when `def` runs. Fix: default to `None` and make a new list inside.

Sources: docs.python.org/3/library/typing · Python FAQ: why are default values shared between objects?

Demo: type hints and docstrings in VS Code

Before: the editor knows nothing

```
def average(scores, drop_lowest=False):
```

After: the editor knows everything

```
def average(
    scores: list[float], drop_lowest: bool = False
) -> float:
    """Return the mean score, optionally ignoring the lowest.

    Args:
        scores: The scores to average. Must not be empty.
        drop_lowest: If True, ignore the lowest score.

    Returns:
        The mean of the scores that count.
    """
```

Hover over `average` : the signature and docstring appear, nicely formatted.

Autocomplete: type `result.` and VS Code lists what a `float` can do.

Red squiggles: `average("90")` is flagged before you run anything.

Google style: a summary line, then **Args**, **Returns**, **Raises**.

Agents read the same hints and docstrings, and write them for you if you ask. You still need to read them.

Sources: Google Python Style Guide 3.8 · VS Code: Python editing

Classes: data plus behaviour

Use a class when data and the operations on it belong together. Otherwise, plain functions.

`@dataclass` writes `__init__`, `__repr__` and `__eq__` from the fields you list.

`field(default_factory=list)`: a new list for every instance. The dataclass version of the mutable-default fix.

`@property`: a computed value that reads like an attribute and never goes stale.

```python
from dataclasses import dataclass, field


@dataclass
class Playlist:
    name: str
    songs: list[str] = field(default_factory=list)

    def add(self, song: str) -> None:
        self.songs.append(song)

    @property
    def size(self) -> int:
        return len(self.songs)


focus = Playlist("Focus")
focus.add("Clair de Lune")
print(focus)       # Playlist(name='Focus', songs=['Clair de Lune'])
print(focus.size)  # 1
```

Docstrings left out to fit the slide.

Sources: docs.python.org/3/library/dataclasses

Exercise 1: fill in the gradebook (7 min)

File: `exercises/gradebook.py` · read each docstring, then write the body

`letter_grade(average)`

- 90+ is A, 80+ B, 70+ C, 60+ D
- anything lower is F

```
letter_grade(59.5) -> "F"
```

`Student.add_score(score)`

- append to `self.scores`
- `ValueError` if the score is outside 0-100

`Student.average`

- a property
- the mean of the scores
- `0.0` if there are none

Check: `uv run exercises/gradebook.py` and compare with the `# expected:` comments

A `NotImplementedError: TODO` on the first run is expected: that's your first task.

Exercise 2: review the AI's code (8 min)

File: `exercises/review.py`

An AI agent wrote this file. It runs without crashing, the type hints and docstrings look great, and it has **three bugs**.

1. Run it and compare with the `# expected:` comments.
2. For each wrong line of output, find the line of code responsible.
3. Fix it, rerun, repeat.

Hint: your editor already flags one of the bugs. The other two you have to read for.

```
uv run exercises/review.py
```

```
['90', '85', '100']
90
['Ada']
['Great work']
['Great work', 'See me after class']
```

What you'll see. Every line here is wrong.

PART 3 · 1:00 - 1:25

How code gets written today

Tools that check your code, and rules your AI agent follows

A project, the way teams set one up

```
my-app/
├── AGENTS.md          rules for AI coding agents
├── README.md          how to set up and run it
├── pyproject.toml     dependencies + tool settings
├── uv.lock            exact versions
├── .python-version
├── .env               secrets: never shared
└── main.py
```

```toml
# pyproject.toml
[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "D"]

[tool.ruff.lint.pydocstyle]
convention = "google"
```

uv: environment and dependencies
Part 1.

Ruff: lint and format
Finds bug patterns, unused imports and missing docstrings, and formats everything the same way. `uvx ruff check .` · `uvx ruff format .`

A type checker
Checks that the type hints agree with the code. In VS Code it's the red squiggles. On the command line: `uvx ty check` (beta).

AGENTS.md: rules for your agent
Plain Markdown. Coding agents read it before they start: Codex, Cursor, GitHub Copilot, Claude Code and more (a few need a setting).

Sources: docs.astral.sh/ruff · docs.astral.sh/ty · agents.md

Demo: let the tools review first

On the original `exercises/review.py`, before any fixes

```
uvx ruff check exercises/review.py
uvx ty check exercises/review.py
uvx ruff format --diff exercises/review.py
```

> Seconds to run, nothing to install, and they never get tired.
> Your agent can run them too, if you tell it to.

Bug from Exercise 2	Caught by
strings instead of floats	type checker
mutable default argument	Ruff (`B006`)
`ranked[: n - 1]`	**only you**

Tools catch the patterns.
You catch the logic.

Sources: Ruff rule B006 · docs.astral.sh/ty

Demo: with and without AGENTS.md

The prompt, both times

> *"Read scores.csv and print each student's average score and letter grade."*

From `starter/AGENTS.md`

```
- Use uv for everything: `uv add` to add
  a dependency, `uv run` to run code.
  Never call pip.
- Type hints and a Google-style
  docstring on every function.
- Run `uvx ruff check .` and fix what
  it reports. Run the code.
```

Watch for

	Without	With
installs packages with	?	?
type hints and docstrings	?	?
runs Ruff	?	?
runs the code and shows output	?	?

Without AGENTS.md you get the agent's habits. With it you get **yours**, every time, without repeating yourself in every prompt.

Exercise 3: your own AGENTS.md, in pairs (8 min)

Next to the repo folder, not inside it:

```
uv init --no-package grades
cd grades
cp ../mcda-python-refresher/starter/* .
```

`cp` works in PowerShell too.

1. Add one rule of your own to `AGENTS.md`, e.g. "print results as a rich table" or "explain your changes in plain English".

2. Give your agent the prompt from the demo.

3. Check its work, below.

Did it follow every rule?

`uv run main.py` : does it run? Ada's average is 91.67, an A

`uvx ruff check .` : clean?

`pyproject.toml` : any new dependencies? Did it ask first?

Your own rule: followed?

Could you explain every line to your partner?

Agent in your editor (Copilot, Cursor, Claude Code, Codex, ...): it reads AGENTS.md itself.

Chat only (ChatGPT, Claude, Gemini): paste AGENTS.md and the prompt; you run the commands.

No AI access: pair with someone who has it.

Verify before you trust

Run it against answers you know

Pick inputs where you already know the output, like Ada's 91.67. "It ran" is not "it's right."

Read it

Let Ruff and the type checker go first, then read what's left. If you can't explain a line, ask the agent to, or rewrite it.

Check new packages exist

AIs sometimes invent package names, and attackers register them ("slopsquatting"). One study found made-up packages in 5.2% to 21.7% of AI code samples. Look it up on pypi.org before `uv add`: its age, downloads, and linked repo.

Keep secrets and private data out

API keys go in `.env`, read from environment variables. Don't paste private data or credentials into public chat tools.

Sources: Spracklen et al., "We Have a Package for You!", USENIX Security 2025 (arXiv 2406.10279)

Checklist for every project

- [ ] A **uv project**: `pyproject.toml` and `uv.lock`, and no more "which pip?"
- [ ] An **AGENTS.md** with your rules, so every agent works your way
- [ ] **Ruff and a type checker** clean before you call it done
- [ ] **Small functions** with type hints and docstrings, and you can explain every line
- [ ] **No secrets in code**: `.env` for keys, read from environment variables

Resources

uv and Ruff

docs.astral.sh/uv
guides for projects, scripts, tools and Python versions

docs.astral.sh/ruff
every lint rule, with examples of the fix

Readable Python and agents

Python typing documentation
every type hint, with examples

Google Python Style Guide: docstrings
Args, Returns, Raises, with examples

agents.md
the AGENTS.md format, and which agents read it

Today's exercises and slides

`github.com/smuotoe/mcda-python-refresher`

Thank you

Questions?

github.com/smuotoe/mcda-python-refresher